Homework Assignment 2

Solutions

Problem 1 [20pt]

While working on this problem, employ functions from the `apply()` family whenever you find the necessity for utilizing loops.

1. Develop a function named `fun1` that takes one parameter, `n`, denoting the number of trials in your simulation. The function should simulate the process of rolling two fair dice and report the outcomes of `n` trials. It should return vectors for the outcomes of both dice, labeled as `Die1` and `Die2`, respectively, along with a vector named `comparison`. The comparison vector is generated using `mapply()` and is a factor with three levels: `"Die1 > Die2"`, `"Die1 < Die2"`, and `"Die1 = Die2"`. For example, if the result of `Die1` is 5 and the result of `Die2` is 4, the comparison should be `"Die1 > Die2"`.

   Test the function with `n = 30`.

```
fun1 <- function(n) {

  Die1 <- sample(1:6, size = n, replace = TRUE)
  Die2 <- sample(1:6, size = n, replace = TRUE)

  compare_dice <- function(Die1, Die2) {
    ifelse(Die1 > Die2, "Die1 > Die2", ifelse(Die1 < Die2, "Die1 < Die2", "Die1 = Die2"))
  }

  comparison <- factor(mapply(compare_dice, Die1, Die2))

  data.frame("trial" = 1:n, Die1, Die2, comparison)
}
```

```
fun1(30)
```

```
   trial Die1 Die2  comparison
1      1    1    1 Die1 = Die2
2      2    4    4 Die1 = Die2
3      3    1    3 Die1 < Die2
4      4    2    6 Die1 < Die2
5      5    5    2 Die1 > Die2
6      6    3    2 Die1 > Die2
7      7    6    6 Die1 = Die2
8      8    2    4 Die1 < Die2
9      9    3    4 Die1 < Die2
10    10    3    4 Die1 < Die2
11    11    1    2 Die1 < Die2
12    12    5    4 Die1 > Die2
13    13    5    1 Die1 > Die2
```

```
14    14    2    6 Die1 < Die2
15    15    6    1 Die1 > Die2
16    16    6    4 Die1 > Die2
17    17    2    1 Die1 > Die2
18    18    1    6 Die1 < Die2
19    19    5    2 Die1 > Die2
20    20    5    3 Die1 > Die2
21    21    1    2 Die1 < Die2
22    22    1    6 Die1 < Die2
23    23    6    6 Die1 = Die2
24    24    5    2 Die1 > Die2
25    25    5    5 Die1 = Die2
26    26    2    2 Die1 = Die2
27    27    2    6 Die1 < Die2
28    28    6    6 Die1 = Die2
29    29    1    6 Die1 < Die2
30    30    4    1 Die1 > Die2
```

2. Construct a function named `fun2` with a single argument, `n`, representing the number of trials in your simulation. The function should simulate the process of rolling an unfair die with the following probability distribution: `"1"` - 0.1, `"2"` - 0.3, `"3"` - 0.2, `"4"` - 0.05, `"5"` - 0.3, `"6"` - 0.05. It should report the results of the `n` trials by returning a frequency table of the outcomes, indicating how many times each outcome (e.g., `"1"`, `"2"`, etc.) occurs.

   Test the function with `n = 100`.

```r
fun2 <- function(n) {
  outcomes <- sample(1:6, size = n, replace = TRUE, prob = c(0.1, 0.3, 0.2, 0.05, 0.3, 0.05))
  counts <- table(outcomes)
  data.frame(cbind("outcome" = 1:6, "count" = counts))
}
```

```r
fun2(100)
```

```
  outcome count
1       1     9
2       2    34
3       3    23
4       4     5
5       5    27
6       6     2
```

3. Create a function named `fun3` that takes a list as input and performs the following operations:

   - Calculate the mean value for each element in the input list.
   - Reorder the elements in the list based on their mean values, arranging them in ascending order. The first element will have the smallest mean value, followed by the second element with the second smallest mean value, and so on.
   - Return the newly rearranged list.

   Test your function with the following list: `list([20, 13, 34, 2], [2, 3], [45, 65, 78], [43, 98, 1, 22], [3, 5, 10], [13, 19, 14])`.

```r
fun3 <- function(list) {

  means <- sapply(list, mean)
  sorted_indices <- order(means)

  list[sorted_indices]
}
```

```r
test_list <- list(
  c(20, 13, 34, 2),
  c(2, 3), c(45, 65, 78),
  c(43, 98, 1, 22),
  c(3, 5, 10),
  c(13, 19, 14)
)

fun3(test_list)
```

```
[[1]]
[1] 2 3

[[2]]
[1]  3  5 10

[[3]]
[1] 13 19 14

[[4]]
[1] 20 13 34  2

[[5]]
[1] 43 98  1 22

[[6]]
[1] 45 65 78
```

4. Create a function called `fun4` with three parameters: `n`, `mu`, and `sigma` (where `n` is the number of randomly generated numbers, `mu` is the mean value of the distribution, and `sigma` is the standard deviation of that distribution). The function performs the following tasks:

   - Generate `n` random numbers from a Normal distribution with mean `mu` and standard deviation `sigma`.
   - Populate an empty matrix based on the following conditions: If `n` is divisible by 2 but not by 3, fill the matrix by rows with 2 rows and `n`/2 columns. If `n` is divisible by 3 but not by 2, fill the matrix by columns with 3 columns and `n`/3 rows. If `n` is divisible by both 2 and 3, fill the matrix by rows with 6 rows and `n`/6 columns. Otherwise, populate the matrix by rows with only one column.
   - Return a vector containing the mean values of each column in the matrix if none of the randomly generated numbers are negative. If some of the generated numbers are negative, return a vector of the median values of each row in the matrix.

   Test the function with parameters (`n = 50`, `mu = 0.5`, `sigma = 2`).

```r
fun4 <- function(n, mu, sigma) {

  random_nums <- rnorm(n, mean = mu, sd = sigma)

  if (n %% 2 == 0 & n %% 3 == 0) {
    mat <- matrix(random_nums, nrow = 6, byrow = TRUE)
  } else if (n %% 2 == 0) {
    mat <- matrix(random_nums, nrow = 2, byrow = TRUE)
  } else if (n %% 3 == 0) {
    mat <- matrix(random_nums, ncol = 3, byrow = FALSE)
  } else {
    mat <- matrix(random_nums, ncol = 1, byrow = TRUE)
  }

  if (all(mat >= 0)) {
    apply(mat, MARGIN = 2, FUN = mean)
  } else {
    apply(mat, MARGIN = 1, FUN = median)
  }
}
```

```r
fun4(n = 50, mu = 0.5, sigma = 2)
```

```
[1] 0.5931606 0.5507657
```

Problem 2 [25pt]

Download the `carseats` dataset from Courseworks and import it into R (ensure to convert all character variables into factors). The dataset offers details on the sales of child car seats across 400 distinct stores, featuring the following variables:

- `Sales` - Unit sales (in thousands) at each location
- `Advertising` - Local advertising budget for company at each location (in thousands of dollars)
- `Population` - Population size in region (in thousands)
- `Price` - Price company charges for car seats at each site
- `ShelveLoc` - A factor with levels `Bad`, `Good`, and `Medium` indicating the quality of the shelving location for the car seats at each site
- `Age` - Average age of the local population
- `Urban` - A factor with levels `No` and `Yes` to indicate whether the store is in an urban or rural location
- `US` - A factor with levels `No` and `Yes` to indicate whether the store is in the US or not

```r
carseats <- read.csv("carseats.csv", stringsAsFactors = TRUE)
```

1. What is the average price the company charges for car seats? How does the average price for car seats in the US compare to that in non-US stores? Additionally, what is the average price for car seats in rural US stores compared to those in rural areas outside of the US?

```r
# average price
mean(carseats$Price)
```

```
[1] 115.795
```

```r
# average price in US vs. non-US stores
aggregate(Price ~ US, FUN = mean, data = carseats)
```

```
   US    Price
1  No 113.9507
2 Yes 116.8101
```

```r
carseats_rural <- carseats[carseats$Urban == "No", ]

# average price in rural US vs. rural non-US
aggregate(Price ~ US, FUN = mean, data = carseats_rural)
```

```
   US    Price
1  No 113.5000
2 Yes 114.4444
```

NOTE: The `tapply()` function also works here. See notes on Problem 2.3 for an extended discussion on the differences between `aggregate()` and `tapply()`.

2. Which store has the highest charge (provide the index number in the dataset)? Conversely, which store has the lowest charge?

```r
# highest charge
which.max(carseats$Price)
```

```
[1] 166
```

```r
# lowest charge
which.min(carseats$Price)
```

```
[1] 43
```

3. What is the median number of unit sales for various quality levels of shelving locations? Now, respond to the same question by incorporating the `Urban` variable, resulting in six combinations.

```r
# median number of unit sales by shelving location quality
aggregate(Sales ~ ShelveLoc, FUN = median, data = carseats)
```

```
  ShelveLoc Sales
1       Bad  5.21
2      Good 10.50
3    Medium  7.38
```

```r
# median number of unit sales by shelving location quality and by urban/rural divide
aggregate(Sales ~ ShelveLoc + Urban, FUN = median, data = carseats)
```

```
  ShelveLoc Urban  Sales
1       Bad    No  5.140
2      Good    No  9.740
3    Medium    No  7.455
4       Bad   Yes  5.295
5      Good   Yes 10.960
6    Medium   Yes  7.370
```

NOTE: You can also use `tapply()` here. A key difference between the two functions is that `aggregate()` returns a data frame, while `tapply()` returns a vector/matrix:

```
tapply(carseats$Sales, carseats$ShelveLoc, FUN = median)
```

```
   Bad   Good Medium
  5.21  10.50   7.38
```

```
tapply(carseats$Sales, list(carseats$ShelveLoc, carseats$Urban), FUN = median)
```

```
          No    Yes
Bad    5.140  5.295
Good   9.740 10.960
Medium 7.455  7.370
```

Also, unlike `aggregate()`, `tapply()` does not allow you to use formulas (e.g. `Sales ~ ShelveLoc + Urban`), so if you want to avoid writing the name of the data frame for every variable, you need to wrap the call to `tapply()` inside a `with()` like this:

```
with(carseats, tapply(Sales, ShelveLoc, FUN = median))
```

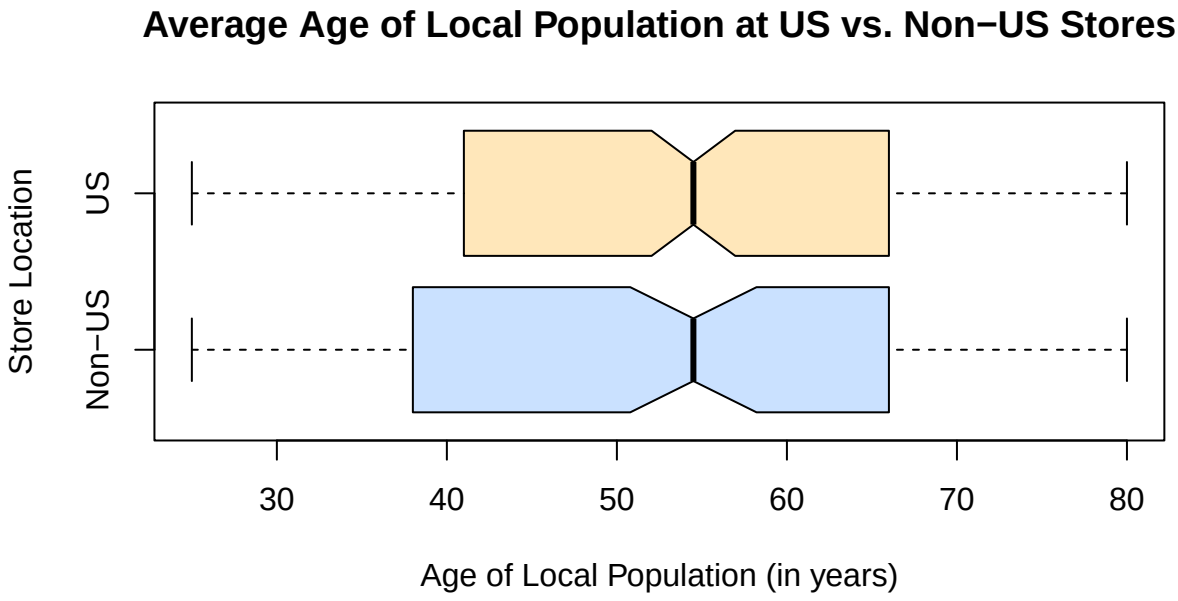
```
   Bad   Good Medium
  5.21  10.50   7.38
```

```
with(carseats, tapply(Sales, list(ShelveLoc, Urban), FUN = median))
```

```
          No    Yes
Bad    5.140  5.295
Good   9.740 10.960
Medium 7.455  7.370
```

4. Contrast the mean age of the local population between US and non-US stores by presenting boxplots. Customize the boxplots by incorporating labels, modifying colors, and adding notches for enhanced clarity.

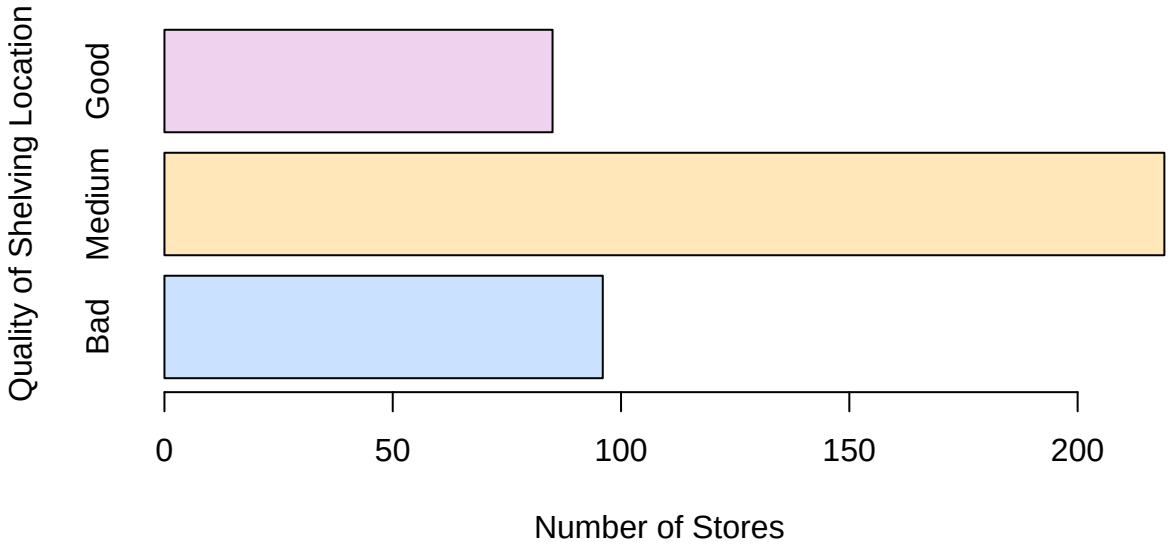
```
boxplot(
  Age ~ US,
  data = carseats,
  horizontal = TRUE,
  col = c("lightsteelblue1", "wheat1"),
  names = c("Non-US", "US"),
  notch = TRUE,
  xlab = "Age of Local Population (in years)",
  ylab = "Store Location",
  main = "Average Age of Local Population at US vs. Non-US Stores"
)
```



5. Provide an overview of the quality of shelving locations using a barplot. Customize the presentation as outlined in the preceding question, excluding notches in this case.

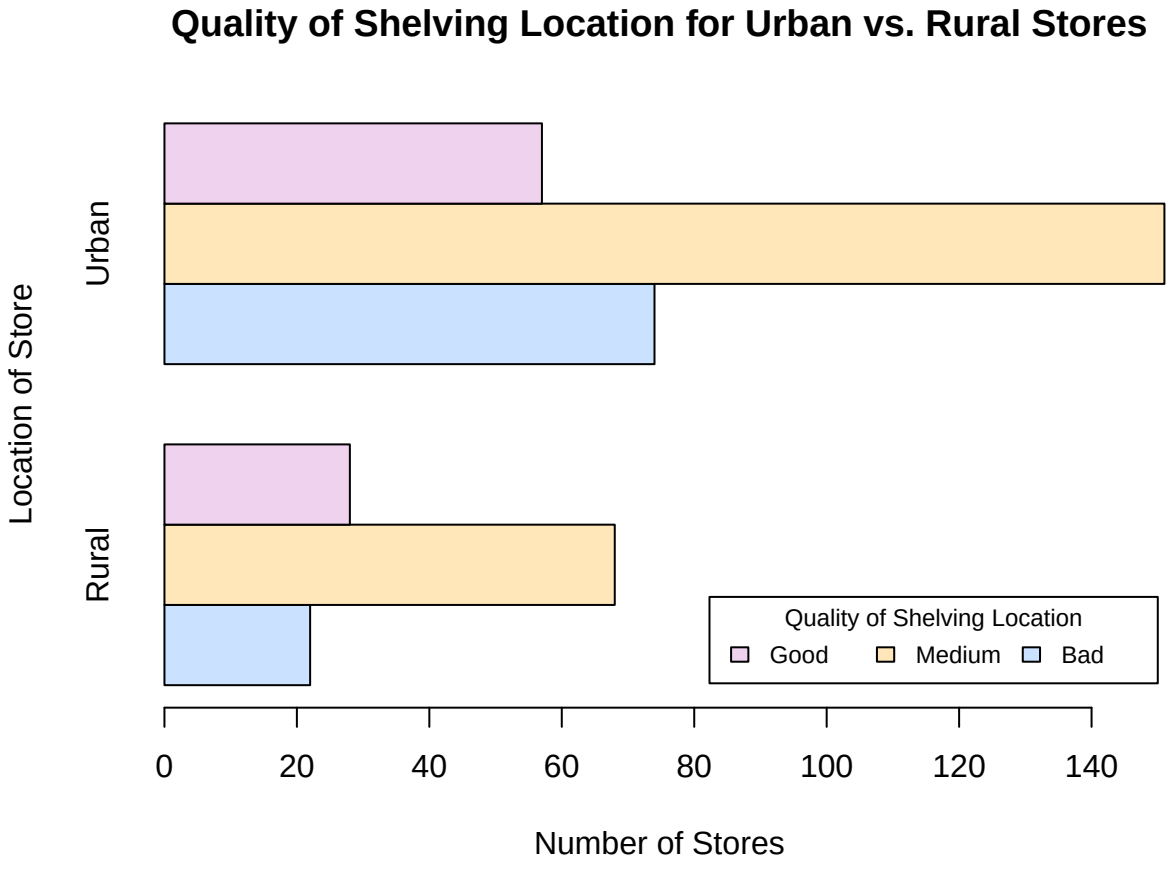
```r
# reorder factor levels
carseats$ShelveLoc <- factor(carseats$ShelveLoc, levels = c("Bad", "Medium", "Good"))

barplot(
  table(carseats$ShelveLoc),
  horiz = TRUE,
  col = c("lightsteelblue1", "wheat1", "thistle2"),
  xlab = "Number of Stores",
  ylab = "Quality of Shelving Location",
  main = "Number of Stores by Shelving Location Quality"
)
```



6. Create side-by-side barplots illustrating the frequency of the `ShelveLoc` variable in both urban and rural areas. Tailor your plot according to the previous discussion.

```
barplot(
  table(carseats$ShelveLoc, carseats$Urban),
  beside = TRUE,
  horiz = TRUE,
  legend.text = levels(carseats$ShelveLoc),
  args.legend = list(
    x = 150,
    y = 2.1,
    horiz = TRUE,
    cex = 0.75,
    title = "Quality of Shelving Location"
  ),
  col = c("lightsteelblue1", "wheat1", "thistle2"),
  xlab = "Number of Stores",
  ylab = "Location of Store",
  names.arg = c("Rural", "Urban"),
  main = "Quality of Shelving Location for Urban vs. Rural Stores"
)
```

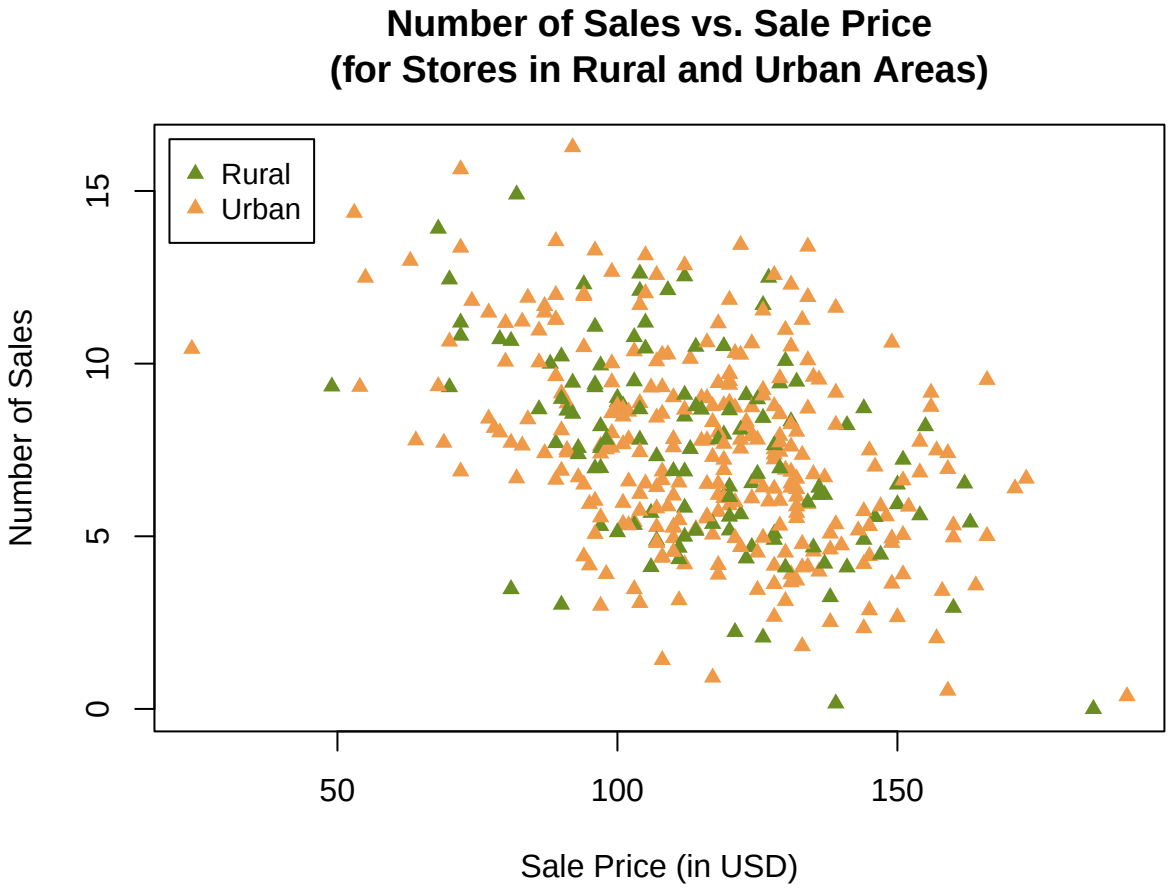


7. Generate a plot of the `Sales` variable against the `Price` variable. Properly label the plot and axes. Adjust the point shape to triangular, with points colored in orange for urban stores and green for rural stores. Include a legend on the plot to indicate the respective areas. Modify the point size to 0.9 for clarity.

```r
colors <- c("olivedrab", "tan2")

plot(
  x = carseats$Price,
  y = carseats$Sales,
  col = colors[carseats$Urban],
  xlab = "Sale Price (in USD)",
  ylab = "Number of Sales",
  main = "Number of Sales vs. Sale Price\n(for Stores in Rural and Urban Areas)",
  pch = 17,
  cex = 0.9
)

legend(x = 20, y = 16.5, legend = c("Rural", "Urban"), cex = 0.9, pch = 17, col = colors)
```



NOTE: By default, R sorts factor levels alphabetically, so when plotting you need to be careful with the order in which you list the colors and the legend labels. The levels of `carseats$Urban` are `"No"` and `"Yes"` (in this order), so the corresponding order of the colors should be green first and orange second, while the corresponding order of the labels should be "Rural" first and "Urban" second. If you write the colors/labels the other way around, your plot will be incorrect.

8. Integrate the histogram of the `Price` variable and the boxplot of the `Population` variable into a single image. Personalize both plots as detailed in part 4, excluding notches in the histogram.

```r
layout(matrix(c(1, 1, 1, 1, 2, 2), nrow = 1, ncol = 6, byrow = TRUE))

hist(
  carseats$Price,
  breaks = 13,
  col = "lightsteelblue1",
  xlab = "Sale Price (in USD)",
  ylab = "Number of Stores",
  main = "Histogram of Sale Price",
  xlim = c(0, round(max(carseats$Price), -2))
)

boxplot(
  carseats$Population,
  col = "wheat1",
  notch = TRUE,
  ylab = "Population (in thousands)",
  main = "Boxplot of Population"
)
```

